Pirtis on Wheels: the receptionist

Pirtis on Wheels rents mobile saunas by phone. A sauna is a trailer. The customer picks it up with their own car. So a booking depends on three things: how many people will use it, what the customer's car can tow, and whether the site has electricity.

You will build the phone receptionist.

The parts

Three parts are ours. Do not change them:

- `pirtis/api`: the rental API, with an in-memory database behind it. Reading returns plain data and never checks the rules. Writing checks every rule and raises an error with a reason code.
- `pirtis/llm`: a chat client and a small tool loop. Copy `.env.example` to `.env` and set `LLM_API_KEY`. The model is always `gpt-4o-mini`, the same for every candidate.
- `pirtis/harness` and `pirtis/cmd`: the test runner. It loads a test case, fills the database, calls your code, and checks what happened.

One part is yours: `pirtis/agent`.

```
# pirtis/agent/agent.py
def decide(t: Transcript, api: api.Client, llm: llm.Client) -> list[Action]: ...


# pirtis/agent/actions.py
handlers: dict[str, Handler]  # action type -> (action, HandlerContext) -> Effect
```

`decide` receives the transcript: every line said so far, with times, whether the last line is still changing, whether the caller has paused, a rough reading of their tone, and the API calls made earlier in the call. It returns what the receptionist does next: `say`, `wait`, `transfer`, `end_call`, or several of them in order.

`handlers` run those actions. The test runner takes every action you return, finds the handler for its type, and records what the handler did. The handlers we ship are one line each: `say` passes the text on, `wait` records the seconds, `transfer` and `end_call` record a reason.

What you may change

Everything between the transcript and the API is yours.

- Add action types, with a handler for each. A hold tone, a text message with a summary, a callback. The runner will run any action that has a handler.
- Change the handlers we ship. If `say` must never speak digits, the `say` handler is one place to make that certain.
- Replace the two tools in `agent.py`, add your own, or wrap `availability` in a tool that already knows what the caller said. Decide what a tool does when the API raises.

- Change or replace the tool loop in `pirtis/llm/loop.py`.

Only two things are fixed on your side: the signatures of `decide` and `handlers`.

What is there now

`agent.py` holds a naive agent. It puts the whole sauna list into the prompt, gives the model two raw tools, and speaks whatever the model answers. It passes the first test case and fails most of the others: it talks over the caller, it never waits, it never transfers, it books with a towing capacity nobody asked for. It is a starting point, not a hint. Replace as much of it as you want.

What we expect

- The receptionist only offers a sauna that can really be booked. If the API answers 422, the receptionist offered something wrong.
- It does not speak while the caller is still speaking. It does not leave a quiet caller waiting.
- It talks like a person on the phone. Short sentences. Numbers as words. One question at a time.
- It passes the call to a human when needed. For example, when the caller is not a known customer; new customers cannot sign up by phone.
- You design the tools and the actions. Decide what your code should do and what the model should do. A new tool with two lines of comment on why is better than a prompt that tries to do the same.
- Every model call costs money and time. On the phone, the caller hears that time as silence. The runner prints calls, tokens and seconds. Most turns need one model call or none.

How to run

Python 3.12 or newer and uv.

```
uv sync                              # once: virtualenv and dependencies
uv run harness                       # all test cases, one time each
uv run harness --id silence          # one test case
uv run harness --runs 3 --need 2     # three times each, two must pass
uv run harness -v                    # also print actions, effects and judge results
uv run harness --trace runs          # also save every model call and a result.json per run
uv run pytest                        # the API's own tests, and the harness's
```

`uv run harness --help` lists every flag. `uv run python -m pirtis.cmd.harness` works too.

A test case checks what happened after your turn: the bookings, the API calls, and what kind of action you returned. When the important thing is what the receptionist said, a judge model reads the reply and checks the statements written in the test case. Two rules are checked every time: no API call may fail (unless the test makes it fail on purpose), and customer data may only be read or changed for a caller who has been identified.

The model is not deterministic. A test case that passed once may fail next time; `--runs 3 --need 2` tells you more. We also have hidden test cases with the same rules but other callers, cars and dates. Make the rules work, not just the test cases.

Where to look

- `../fixtures/*.yaml`: the test cases. Each one has the expected result and the call so far. Read a few before you start. `../fixtures/seeds/base.json` has the saunas, the customers and the existing bookings.
- `pirtis/transcript/types.py`: what a transcript looks like, and helper functions for actions.
- `pirtis/api/client.py` and `types.py`: every API method and error code, with comments.
- `pirtis/llm/`: the tool loop, JSON output, and a fake model for your own tests.
- `pirtis/harness/fixture.py`, `assert_.py`, `judge.py`: exactly what the tests check.

Data that crosses a boundary keeps the wire spelling: the transcript, the API records, the fixtures and the chat messages are dataclasses with `snake_case` fields, the same as the YAML and JSON they come from. Two keys are Python keywords and so get renamed on the dataclass only: `from` is `from_` on the query types, and a verdict's `pass` is `passed`.

Submitting

You may use any tools you like, including AI. After you submit, we will go through the code with you. Be ready to explain every line, also the lines your tools wrote.

Send us the repo. Plan for two to three hours. If you run out of time, write a short note about what you would do next.

Our solution

The receptionist is built in `pirtis/agent/`. Around it, never imported by it:

- `pirtis/playground/`: a caller simulator and an admin console
- `pirtis/ops/`: the data layer, the gate job, evaluation tools

Start with the root README. How the evaluation and monitoring numbers are computed: docs/data-science.md. Everyday commands are in the `Makefile`:

- `make check`
- `make gate`
- `make eval-extract`
- `make playground`